Software Engineering（软件工程）

不考例子 只考概念

Software and Software Engineering

nature of software——defining software

software is:**指令 数据结构 文档**

- instructions(computer programs)
- data structures
- documents

what is software

软件和硬件的区别：

- not manufactured in classical sense
- not "wear out"，but deteriorate（不会磨损，但会恶化、生命周期）
- most custom-built

software application

- System software
- Application software
- Engineering/Scientific software
- Embedded software
- Product-line software
- Web/Mobile applications
- AI software (robotics, neural nets, game playing).

why must software change

- **adapted** to meet the needs （适应）
- **enhanced** to implement new business requirements （增强）
- **extended** to make it interoperable （扩展）
- **re-architected** to make a network environment （重构）

process framework activities 5个阶段

- communication
  - understand the problem
- planning
  - tasks, denpendences, schedule, resources
- modeling
  - analysis of requirement
  - design
- construction
  - coding
  - testing
- deployment

Process Models

- Generic Process Model 惯用过程模型
  - 惯用过程模型是为了改变软件开发的混乱状态，促使软件开发更加有序。

Agility and Process

What is Agility

- effective(rapid and adaptive) response to change
- effective communication among all stakeholders
- drawing customer onto team 将客户吸引到团队中
- Organizing a team so that it is in control of the work performed. ?????
- rapid, incremental delivery of software 快速、增量 交付

What is an Agile Process

- driven by customer description of what is required (scenarios 场景)
- customer feedback(反馈) is frequent and acted on
- recognizes that plans are short-lived
- iteratively(迭代) develop
- Delivers multiple 'software increments' as executable prototypes. ????///
- adapt as project or technical changes occur

Scrum Framework

敏捷开发——Scrum Framework

Characteristics of Agile Process Models

- not suitable for large high-risk or mission critical projects
- minimal rules and minimal documentations
- continuous involvement of testers
- easy to accommodate product changes
- depends heavily on stakeholder interaction
- easy to manage
- early delivery of partial solutions
- informal risk management
- built-in continuous process improvement

Recommended Process Model

Characteristics of Agile Process Models

敏捷开发的特点

- not suitable for high-risk or mission project
- minimal rules and document
- continuous involvement of testers
- esay to adapt product change
- depends heavily on stakeholder interaction
- esay to manage
- early delivery of partial solutions 尽早交付解决方案
- informal risk management
- built-in continuous process improvement 内置流程持续改进

First Prototype Guidelines

第一产品原型指南

- transition from paper prototype to software design
- prototype a user interface
- create a virtual prototype
- add input and output to prototype
- engineer algorithms
- test prototype
- prototype with deployment in mind

Prototype Evaluation

原型评估

-

Go No Go Decision

是否通过

Recommended Prototype Evolutionary Process

推荐的原型演变过程

- requirement engineering
- preliminary architectural design 初步的架构设计
- estimate required project resources
- construct first prototype
- evaluate prototype
- go,no go decision
- evolve system 进化
- release prototype
- maintain software

Human Aspects of Software Engineering

Traits of Successful Software Engineers

优秀软件工程师的特征

- sense of individual responsibility
- aware of the need of team members and stockholders
- be honest
- resilient under pressure 抗压
- sense of fairness 公平感
- attention to detail
- Pragmatic adapting software engineering practices based on the circumstances at hand 根据手头的情况务实地调整软件工程实践

Effective Software Team Attributes

高效软件团队的属性

- sense of purpose
- sense of involvement

- sense of trust
- sense of improvement
- diversity of team members skill sets

使命感。
参与感。
信任感。
进步感。
团队成员技能组合的多样性。

Principles that Guide Practice

Understanding Requirements

Requirements Engineering

起始、获取、细化、协商、规格说明、确认、管理

- Inception 起始
- Elicitation 获取
    - Collaborative Requirements Gathering 协作需求收集
- Elaboration 细化
    - Use Case Definition
    - Analysis Model Elements(requirement model)
    - U M L Use Case Diagram
    - U M L Class Diagram
    - U M L State Diagram
- Negotiation 协商
    - Negotiating Requirements
- Specification 规范
    - Requirements Monitoring
- Validation 确认
    - Validating Requirements
- Requirements management 管理

Requirements Modeling – A Recommended Approach

Requirements Models

需求模型

- Scenario-based models 基于场景建模
  - UML: actors and profiles
  - user case
- Class-oriented models 基于类建模
  - Class-responsibility-collaborator (C R C) 类-职责-协作者建模
  - classes and objects, attributes, operations, C R C models, U M L class diagrams
- Behavioral models 行为模型
  - Identifying Events
  - State Diagram，Activity Diagram，Swimlane Diagram
- Data models 数据模型
- Flow-oriented models 面向流模型

Architectural Design – A Recommended Approach

Data Centered Architecture

Data Flow Architecture

Call Return Architecture

Object-Oriented Architecture

Layered Architecture

Understanding Requirements

Component-Level Design

Cohesion 内聚性

Coupling 耦合性

高内聚低耦合

User Experience Design

Golden Rules

- Place User in Control 用户操纵控制
- Reduce User's Memory Load 减少用户的记忆负担
- Make Interface Consistent 保持界面一致

Quality Concepts

what is quality

- quality of design 设计质量
- quality of conformance 一致性质量
- user satisfaction 用户满意度
    - compliant product 合规的产品
    - good quality 好的质量
    - delivery within budget and shedule 按时按预算交付

Reviews – A Recommended Approach

reviews

what are they?

- meeting conducted by technical people
- technical assessment of software product
- software quality assurance mechanism 软件质量保证机制
- training ground 训练场

what they are not!

- project summary or progress assessment
- meeting intended solely to impart information
- mechanism for political or personal reprisal!

Error—a quality problem found before the software is released to end users

Defect缺陷—a quality problem found only after the software has been released to end-users

Software Quality Assurance(SQA)

SQA Goals

- requirement quality
- design quality
- code quality
- effective quality control

Software Testing(Component Level)

Verification and Validation

验证和确认

- Verification
  - specific function
  - Are we building the product right?
- Validation
  - customer requirement
  - Are we building the right product?

Testing

from small to big

- unit test
  - code
- integration test
  - design architecture
- validation test
  - requirement
- system test

stubs

Software Testing(Integration Level)

Testing Fundamentals

Attributes of a good test：a good test has

- high probability of finding errors
- not redundant
- be of breed （同类最佳）
- neither too simple nor too complex

White Box Testing

according to source codes to test

也是一种集成测试

Integration Testing

- Top-Down Integration
  - dfs
  - bfs
- Bottom-Up Integration
- Smoke testing
- Regression Testing

- O O Integration Testing
- Validation Testing
    - user requirements
    - deficiency list（缺陷清单）
    - configuration review（配置回顾）